

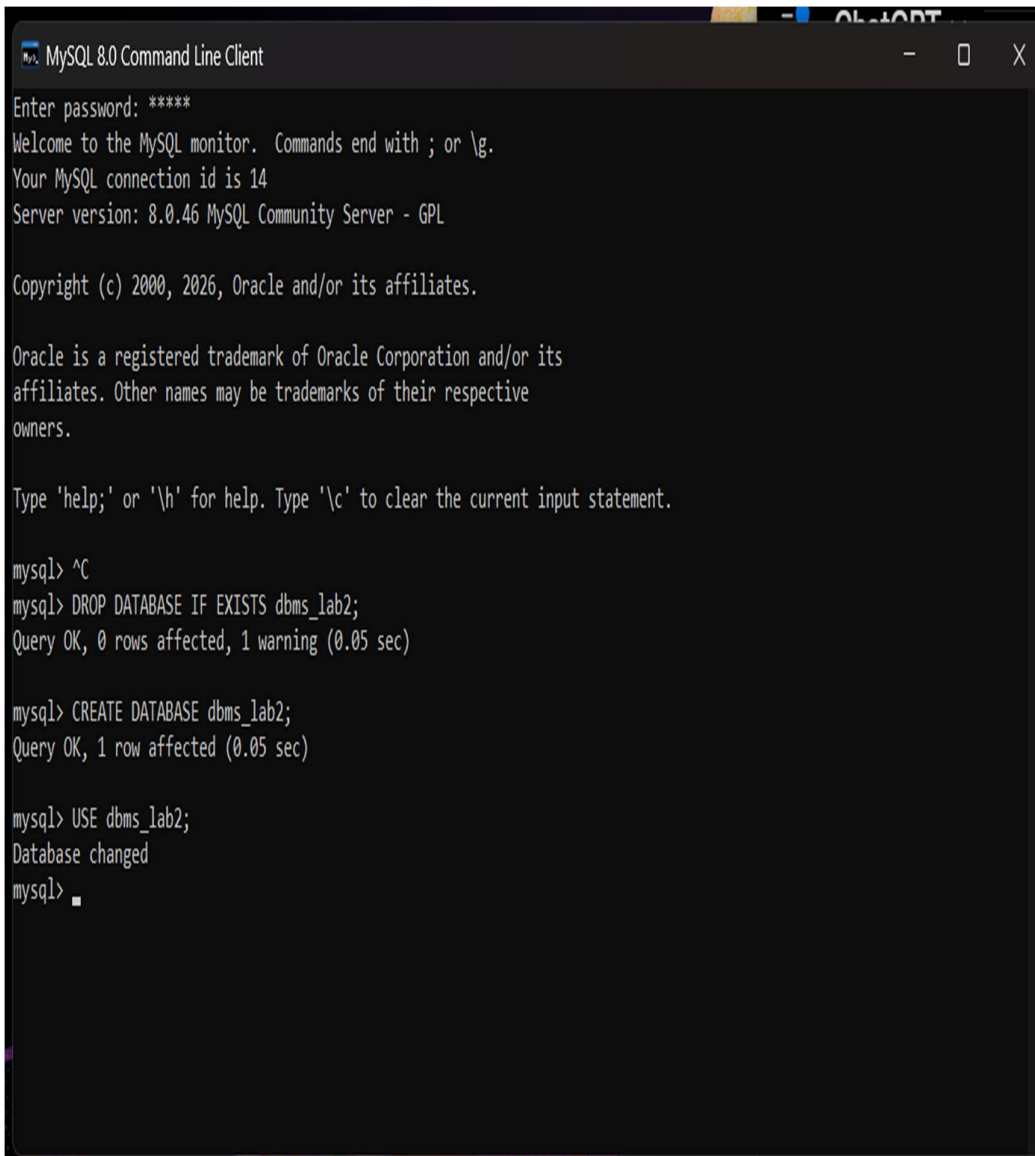
DBMS Lab: SQL Joins and Transactions

Monish Putti

2520030395

Using the following tables:

- Employees
- Departments
- Projects
- Customers
- Orders
- Products
- Order_Details

A screenshot of a MySQL 8.0 Command Line Client window. The window title is "MySQL 8.0 Command Line Client". The terminal shows the following text: "Enter password: *****", "Welcome to the MySQL monitor. Commands end with ; or \g.", "Your MySQL connection id is 14", "Server version: 8.0.46 MySQL Community Server - GPL", "Copyright (c) 2000, 2026, Oracle and/or its affiliates.", "Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.", "Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.", "mysql> ^C", "mysql> DROP DATABASE IF EXISTS dbms_lab2;", "Query OK, 0 rows affected, 1 warning (0.05 sec)", "mysql> CREATE DATABASE dbms_lab2;", "Query OK, 1 row affected (0.05 sec)", "mysql> USE dbms_lab2;", "Database changed", "mysql>".

```
MySQL 8.0 Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ^C
mysql> DROP DATABASE IF EXISTS dbms_lab2;
Query OK, 0 rows affected, 1 warning (0.05 sec)

mysql> CREATE DATABASE dbms_lab2;
Query OK, 1 row affected (0.05 sec)

mysql> USE dbms_lab2;
Database changed
mysql>
```

```
MySQL 8.0 Command Line Client
mysql> USE dbms_lab2;
Database changed
mysql> CREATE TABLE Employees (
  ->   emp_id INT PRIMARY KEY,
  ->   emp_name VARCHAR(50),
  ->   email VARCHAR(100),
  ->   dept_id INT,
  ->   manager_id INT,
  ->   salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Departments (
  ->   dept_id INT PRIMARY KEY,
  ->   dept_name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Projects (
  ->   project_id INT PRIMARY KEY,
  ->   project_name VARCHAR(100),
  ->   dept_id INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Customers (
  ->   customer_id INT PRIMARY KEY,
  ->   customer_name VARCHAR(50),
  ->   email VARCHAR(100),
  ->   city VARCHAR(50)
```

```
MySQL 8.0 Command Line Client
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Products (
  ->   product_id INT PRIMARY KEY,
  ->   product_name VARCHAR(100),
  ->   price DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Orders (
  ->   order_id INT PRIMARY KEY,
  ->   customer_id INT,
  ->   order_date DATE
  -> );
Query OK, 0 rows affected (0.08 sec)

mysql> CREATE TABLE Orders (
  ->   order_id INT PRIMARY KEY,
  ->   customer_id INT,
  ->   order_date DATE
  -> );
ERROR 1050 (42S01): Table 'orders' already exists
mysql> CREATE TABLE Order_Details (
  ->   order_detail_id INT PRIMARY KEY,
  ->   order_id INT,
  ->   product_id INT,
  ->   quantity INT
  -> );
Query OK, 0 rows affected (0.09 sec)
```

```
MySQL 8.0 Command Line Client

mysql> INSERT INTO Employees VALUES
  -> (101,'Rahul','rahul@gmail.com',1,NULL,70000),
  -> (102,'Priya','priya@gmail.com',1,101,50000),
  -> (103,'Amit','amit@gmail.com',2,101,65000),
  -> (104,'Neha','neha@gmail.com',1,101,80000),
  -> (105,'Kiran','kiran@gmail.com',3,103,90000),
  -> (106,'Suresh','suresh@gmail.com',4,103,28000);
Query OK, 6 rows affected (0.05 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Projects VALUES
  -> (1,'Library Management',1),
  -> (2,'College ERP',2),
  -> (3,'Smart City',3);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Customers VALUES
  -> (1,'Arjun','arjun@gmail.com','Hyderabad'),
  -> (2,'Meena','meena@gmail.com','Vijayawada'),
  -> (3,'Ravi','ravi@gmail.com','Guntur'),
  -> (4,'John','john@gmail.com','Warangal');
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Products VALUES
  -> (1,'Laptop',50000),
  -> (2,'Mouse',500),
  -> (3,'Keyboard',1200),
```

```
MySQL 8.0 Command Line Client

  -> (4,'Monitor',10000);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Orders VALUES
  -> (101,1,'2026-08-01'),
  -> (102,2,'2026-08-02'),
  -> (103,1,'2026-08-03'),
  -> (104,3,'2026-08-04');
Query OK, 4 rows affected (0.05 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Order_Details VALUES
  -> (1,101,1,1),
  -> (2,101,2,2),
  -> (3,102,3,1),
  -> (4,103,4,1),
  -> (5,104,1,2);
Query OK, 5 rows affected (0.06 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Departments;
+-----+-----+
| dept_id | dept_name |
+-----+-----+
| 1 | CSE |
| 2 | ECE |
| 3 | MECH |
| 4 | CIVIL |
+-----+-----+
```

```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM Departments;
+-----+-----+
| dept_id | dept_name |
+-----+-----+
| 1 | CSE |
| 2 | ECE |
| 3 | MECH |
| 4 | CIVIL |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Employees;
+-----+-----+-----+-----+-----+-----+
| emp_id | emp_name | email | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+-----+
| 101 | Rahul | rahul@gmail.com | 1 | NULL | 70000.00 |
| 102 | Priya | priya@gmail.com | 1 | 101 | 50000.00 |
| 103 | Amit | amit@gmail.com | 2 | 101 | 65000.00 |
| 104 | Neha | neha@gmail.com | 1 | 101 | 80000.00 |
| 105 | Kiran | kiran@gmail.com | 3 | 103 | 90000.00 |
| 106 | Suresh | suresh@gmail.com | 4 | 103 | 28000.00 |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> SELECT * FROM Projects;
+-----+-----+-----+
| project_id | project_name | dept_id |
+-----+-----+-----+
| 1 | Library Management | 1 |
+-----+-----+-----+
```

```
MySQL 8.0 Command Line Client

6 rows in set (0.00 sec)

mysql> SELECT * FROM Projects;
+-----+-----+-----+
| project_id | project_name | dept_id |
+-----+-----+-----+
| 1 | Library Management | 1 |
| 2 | College ERP | 2 |
| 3 | Smart City | 3 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Customers;
+-----+-----+-----+-----+
| customer_id | customer_name | email | city |
+-----+-----+-----+-----+
| 1 | Arjun | arjun@gmail.com | Hyderabad |
| 2 | Meena | meena@gmail.com | Vijayawada |
| 3 | Ravi | ravi@gmail.com | Guntur |
| 4 | John | john@gmail.com | Warangal |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Products;
+-----+-----+-----+
| product_id | product_name | price |
+-----+-----+-----+
| 1 | Laptop | 50000.00 |
| 2 | Mouse | 500.00 |
| 3 | Keyboard | 1200.00 |
+-----+-----+-----+
```

```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM Products;
+-----+-----+-----+
| product_id | product_name | price |
+-----+-----+-----+
| 1 | Laptop | 50000.00 |
| 2 | Mouse | 500.00 |
| 3 | Keyboard | 1200.00 |
| 4 | Monitor | 10000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Orders;
+-----+-----+-----+
| order_id | customer_id | order_date |
+-----+-----+-----+
| 101 | 1 | 2026-08-01 |
| 102 | 2 | 2026-08-02 |
| 103 | 1 | 2026-08-03 |
| 104 | 3 | 2026-08-04 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Order_Details;
```

DML Operations:

1. Insert a new employee into the Employees table.

```
mysql> INSERT INTO Employees
-> VALUES (107, 'Ramesh', 'ramesh@gmail.com', 2, 103, 55000);
Query OK, 1 row affected (0.05 sec)

mysql> 
```

2. Update the salary of employees working in the CSE department by 10%.

```
mysql> UPDATE Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> SET e.salary = e.salary * 1.10
-> WHERE d.dept_name = 'CSE';
Query OK, 3 rows affected (0.05 sec)
Rows matched: 3 Changed: 3 Warnings: 0

mysql> 
```

3. Delete employees whose salary is below ₹30,000.

```
mysql> DELETE FROM Employees
-> WHERE salary < 30000;
Query OK, 1 row affected (0.05 sec)

mysql> 
```

4. Display all employee details.

```
mysql> SELECT * FROM Employees;
+-----+-----+-----+-----+-----+-----+
| emp_id | emp_name | email          | dept_id | manager_id | salary |
+-----+-----+-----+-----+-----+-----+
| 101    | Rahul    | rahul@gmail.com | 1       | NULL       | 77000.00 |
| 102    | Priya    | priya@gmail.com | 1       | 101        | 55000.00 |
| 103    | Amit     | amit@gmail.com  | 2       | 101        | 65000.00 |
| 104    | Neha     | neha@gmail.com  | 1       | 101        | 88000.00 |
| 105    | Kiran    | kiran@gmail.com | 3       | 103        | 90000.00 |
| 107    | Ramesh   | ramesh@gmail.com | 2       | 103        | 55000.00 |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql> _
```

JOIN Queries:

5. Display employee names along with department names.

```
mysql> SELECT e.emp_name,
->      d.dept_name
-> FROM Employees e
-> INNER JOIN Departments d
-> ON e.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Rahul    | CSE       |
| Priya    | CSE       |
| Amit     | ECE       |
| Neha     | CSE       |
| Kiran    | MECH      |
| Ramesh   | ECE       |
+-----+-----+
6 rows in set (0.00 sec)

mysql> _
```

6. Display employees who do not belong to any department.

```
mysql> SELECT e.emp_name
-> FROM Employees e
-> LEFT JOIN Departments d
-> ON e.dept_id = d.dept_id
-> WHERE d.dept_id IS NULL;
Empty set (0.00 sec)
```

7. Display all departments even if they have no employees.

```
mysql> SELECT d.dept_name,
->      e.emp_name
-> FROM Departments d
-> LEFT JOIN Employees e
-> ON d.dept_id = e.dept_id;
+-----+-----+
| dept_name | emp_name |
+-----+-----+
| CSE       | Neha     |
| CSE       | Priya    |
| CSE       | Rahul    |
| ECE       | Ramesh   |
| ECE       | Amit     |
| MECH      | Kiran    |
| CIVIL     | NULL     |
+-----+-----+
7 rows in set (0.00 sec)

mysql>
```

8. Display employee and manager names using SELF JOIN.

```
mysql> SELECT e.emp_name AS Employee,
->      m.emp_name AS Manager
-> FROM Employees e
-> LEFT JOIN Employees m
-> ON e.manager_id = m.emp_id;
+-----+-----+
| Employee | Manager |
+-----+-----+
| Rahul    | NULL    |
| Priya    | Rahul   |
| Amit     | Rahul   |
| Neha     | Rahul   |
| Kiran    | Amit    |
| Ramesh   | Amit    |
+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

Aggregate Functions:

9. Find the total number of employees.

```
mysql> SELECT COUNT(*) AS Total_Employees
-> FROM Employees;
+-----+
| Total_Employees |
+-----+
|                6 |
+-----+
1 row in set (0.04 sec)

mysql>
```

10. Find department-wise average salary.

```
mysql> SELECT d.dept_name,
->         AVG(e.salary) AS Average_Salary
-> FROM Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY d.dept_name;
+-----+-----+
| dept_name | Average_Salary |
+-----+-----+
| CSE       | 73333.333333   |
| ECE       | 60000.000000   |
| MECH      | 90000.000000   |
+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

11. Display departments where average salary exceeds ₹60,000.

```
mysql> SELECT d.dept_name,
->         AVG(e.salary) AS Average_Salary
-> FROM Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY d.dept_name
-> HAVING AVG(e.salary) > 60000;
+-----+-----+
| dept_name | Average_Salary |
+-----+-----+
| CSE       | 73333.333333   |
| MECH      | 90000.000000   |
+-----+-----+
2 rows in set (0.00 sec)

mysql> _
```

12. Display the highest and lowest salary.

```
mysql> SELECT
-> MAX(salary) AS Highest_Salary,
-> MIN(salary) AS Lowest_Salary
-> FROM Employees;
+-----+-----+
| Highest_Salary | Lowest_Salary |
+-----+-----+
|          90000.00 |          55000.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Subqueries:

13. Display employees earning more than the average salary.

```
mysql> SELECT emp_name,
-> salary
-> FROM Employees
-> WHERE salary >
-> (
-> SELECT AVG(salary)
-> FROM Employees
-> );
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 77000.00 |
| Neha     | 88000.00 |
| Kiran    | 90000.00 |
+-----+-----+
3 rows in set (0.04 sec)

mysql> █
```

14. Find employees working in the same department as Rahul.

```
mysql> SELECT emp_name
-> FROM Employees
-> WHERE dept_id =
-> (
-> SELECT dept_id
-> FROM Employees
-> WHERE emp_name='Rahul'
-> );
+-----+
| emp_name |
+-----+
| Rahul    |
| Priya    |
| Neha     |
+-----+
3 rows in set (0.00 sec)

mysql>
```

15. Display the employee receiving the highest salary.

```
mysql> SELECT emp_name,
-> salary
-> FROM Employees
-> WHERE salary =
-> (
-> SELECT MAX(salary)
-> FROM Employees
-> );
+-----+-----+
| emp_name | salary |
+-----+-----+
| Kiran    | 90000.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

16. Display departments having more than five employees.

```
mysql> SELECT d.dept_name,
->         COUNT(*) AS Employee_Count
-> FROM Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY d.dept_name
-> HAVING COUNT(*) > 5;
Empty set (0.00 sec)

mysql>
```

17. Find the second-highest salary using a window function.

```
mysql> SELECT emp_name,
->         salary
-> FROM
-> (
->     SELECT emp_name,
->            salary,
->            DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
->     FROM Employees
-> ) t
-> WHERE rnk = 2;
+-----+-----+
| emp_name | salary |
+-----+-----+
| Neha     | 88000.00 |
+-----+-----+
1 row in set (0.04 sec)

mysql>
```

18. Display employees earning above their department's average salary.

```
mysql> SELECT e.emp_name,
->         e.salary,
->         d.dept_name
-> FROM Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> WHERE e.salary >
-> (
-> SELECT AVG(salary)
-> FROM Employees
-> WHERE dept_id = e.dept_id
-> );
```

emp_name	salary	dept_name
Rahul	77000.00	CSE
Amit	65000.00	ECE
Neha	88000.00	CSE

```
3 rows in set (0.00 sec)

mysql> █
```

19. Find customers who have never placed an order.

```
mysql> SELECT customer_name
-> FROM Customers c
-> LEFT JOIN Orders o
-> ON c.customer_id = o.customer_id
-> WHERE o.customer_id IS NULL;
```

customer_name
John

```
1 row in set (0.00 sec)

mysql> █
```

20. Display the top-selling product.

```
mysql> SELECT p.product_name,
->         SUM(od.quantity) AS TotalSold
-> FROM Products p
-> JOIN Order_Details od
-> ON p.product_id = od.product_id
-> GROUP BY p.product_name
-> ORDER BY TotalSold DESC
-> LIMIT 1;
```

```
+-----+-----+
| product_name | TotalSold |
+-----+-----+
| Laptop      |          3 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> █
```

21. Find departments with the maximum number of employees.

```
mysql> SELECT d.dept_name,
->         COUNT(*) AS Employee_Count
-> FROM Employees e
-> JOIN Departments d
-> ON e.dept_id = d.dept_id
-> GROUP BY d.dept_name
-> ORDER BY Employee_Count DESC
-> LIMIT 1;
```

```
+-----+-----+
| dept_name | Employee_Count |
+-----+-----+
| CSE      |          3 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

22. Display monthly sales using window functions.

```
mysql> SELECT MONTH(o.order_date) AS Month,
->         SUM(od.quantity * p.price) AS MonthlySales
-> FROM Orders o
-> JOIN Order_Details od
-> ON o.order_id = od.order_id
-> JOIN Products p
-> ON od.product_id = p.product_id
-> GROUP BY MONTH(o.order_date);
```

```
+-----+-----+
| Month | MonthlySales |
+-----+-----+
|      8 |    162200.00 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> █
```

23. Rank customers based on total purchase amount.

```
mysql> SELECT customer_name,  
->      SUM(od.quantity * p.price) AS TotalPurchase,  
->      DENSE_RANK() OVER  
->      (ORDER BY SUM(od.quantity * p.price) DESC) AS Ranking  
-> FROM Customers c  
-> JOIN Orders o  
-> ON c.customer_id = o.customer_id  
-> JOIN Order_Details od  
-> ON o.order_id = od.order_id  
-> JOIN Products p  
-> ON od.product_id = p.product_id  
-> GROUP BY customer_name;  
  
+-----+-----+-----+  
| customer_name | TotalPurchase | Ranking |  
+-----+-----+-----+  
| Ravi          | 100000.00    | 1       |  
| Arjun         | 61000.00     | 2       |  
| Meena        | 1200.00      | 3       |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>
```

24. Display employees whose salary is greater than their manager's salary.

```
mysql> SELECT e.emp_name,  
->      e.salary,  
->      m.emp_name AS Manager,  
->      m.salary AS ManagerSalary  
-> FROM Employees e  
-> JOIN Employees m  
-> ON e.manager_id = m.emp_id  
-> WHERE e.salary > m.salary;  
  
+-----+-----+-----+-----+  
| emp_name | salary | Manager | ManagerSalary |  
+-----+-----+-----+-----+  
| Neha     | 88000.00 | Rahul  | 77000.00      |  
| Kiran    | 90000.00 | Amit   | 65000.00      |  
+-----+-----+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> _
```

25. Find duplicate email addresses using GROUP BY and HAVING.

```
mysql> SELECT email,  
->      COUNT(*) AS Total  
-> FROM Employees  
-> GROUP BY email  
-> HAVING COUNT(*) > 1;  
Empty set (0.00 sec)  
  
mysql> _
```

